

UCAB – Cab Booking System

Full Stack Development (MERN)

Project Documentation

Submitted By

Mohammed Farhana

Department of

Electronics and Communication Engineering

College Name

Vasireddy Venkatadri institute of Technology

Academic Year

2026–2027

Table of Content

S.No	CONTENTS	PAGE.NO
1	Introduction	1
2	Project Overview	2-3
3	Architecture	4-8
4	Setup Instructions	9-10
5	Folder Structure	11
6	Running the Application	12
7	API Documentation	13
8	Authentication	14
9	User Interface	15-18
10	Testing	19
11	Screenshots	20-25
12	Known Issues	26
13	Conclusion	27

Introduction:

UCAB (Universal Cab Booking System) is a web-based cab booking application developed using the MERN Stack, which consists of MongoDB, Express.js, React.js, and Node.js. The project is designed to provide a simple, secure, and efficient platform for users to book cabs online while allowing drivers to manage ride requests and administrators to monitor the overall system.

The main objective of this project is to eliminate the difficulties involved in traditional cab booking methods by providing a digital platform where users can register, log in, view available cabs, book rides, and monitor their booking status. Drivers can log in to view assigned bookings, accept or reject ride requests, and update the ride status from "Pending" to "Accepted", "Started", and "Completed". Administrators have complete control over the system through an admin dashboard where they can manage users, cabs, bookings, and view overall statistics.

The application follows the Model-View-Controller (MVC) architecture, which separates the user interface, business logic, and database operations, making the project easy to maintain and scalable. React.js is used to create an interactive frontend, Express.js and Node.js handle the backend APIs, and MongoDB stores all application data such as users, drivers, cars, and bookings.

This project demonstrates the practical implementation of full-stack web development concepts including authentication using JSON Web Tokens (JWT), RESTful APIs, CRUD operations, MongoDB database management, and responsive user interfaces. The system provides a complete workflow starting from user registration to successful ride completion, making it an effective solution for online cab booking management.

Project Overview:

Purpose

The **UCAB – MERN Stack Cab Booking System** is a web-based application developed to provide a simple, secure, and efficient online cab booking platform. It allows users to register, log in, book cabs, and view their booking history. Drivers can manage ride requests and update ride status, while administrators can manage users, cabs, bookings, and monitor the system through an admin dashboard.

The application is developed using the **MERN Stack (MongoDB, Express.js, React.js, and Node.js)**, providing a responsive and user-friendly interface with secure authentication and efficient database management.

Key Features:

The UCAB Cab Booking System provides the following features:

User Module-

- User Registration and Login
- View Available Cabs
- Book a Cab
- View Booking History
- View Ride Status

Driver Module-

- Driver Login
- View Assigned Ride Requests
- Accept or Reject Ride Requests
- Update Ride Status (Pending, Accepted, Started, Completed)

Admin Module-

- Admin Login
- Admin Dashboard
- Dashboard Statistics

- Manage Users
- Manage Cabs
- Add New Cars
- View All Bookings
- Monitor Completed Rides

General Features

- Secure Authentication using JSON Web Tokens (JWT)
- RESTful API Integration
- MongoDB Database Management
- Responsive User Interface using React.js
- CRUD Operations for Users, Cars, and Bookings
- Role-Based Access Control
- Dashboard Analytics
- Real-Time Booking Status Updates

Project Goals

The primary goals of the UCAB Cab Booking System are:

- To provide a digital platform for easy cab booking.
- To reduce manual effort involved in transportation management.
- To provide separate dashboards for Users, Drivers, and Administrators.
- To ensure secure user authentication using JWT.
- To maintain booking information efficiently using MongoDB.
- To provide an easy-to-use interface with a responsive design.
- To demonstrate the implementation of Full Stack Web Development using the MERN Stack.

Architecture:

The UCAB Cab Booking System follows a **four-layer architecture** consisting of the Client Layer, API Layer, Service Layer, and Data Access Layer. This architecture ensures proper separation of concerns, making the application modular, scalable, secure, and easy to maintain. Each layer has a specific responsibility and communicates with the next layer to process user requests efficiently.

Frontend Architecture (React.js)-

The frontend of the UCAB application is developed using **React.js**, a JavaScript library for building interactive user interfaces. React provides a component-based architecture where the application is divided into reusable components, improving code maintainability and performance.

The frontend is responsible for displaying information to users and collecting user inputs. It communicates with the backend through REST APIs using HTTP requests.

The frontend includes the following pages:

- Home Page
- User Registration
- User Login
- Driver Login
- Admin Login
- Available Cabs
- Book Cab
- My Bookings
- Driver Dashboard
- Driver Bookings
- Admin Dashboard
- Manage Users
- Manage Cabs
- Add New Car
- View Bookings

React Router is used for page navigation, while React Hooks such as **useState** and **useEffect** are used for state management and API communication.

Responsibilities of the Frontend

- Display user-friendly interfaces
- Collect user information
- Send requests to backend APIs
- Display booking details
- Display dashboard statistics
- Display ride status
- Handle user navigation

Back end Architecture (Node.js and Express.js)-

The backend of the application is developed using **Node.js** and **Express.js**. Express.js acts as the web framework that handles HTTP requests and responses. The backend contains all business logic and communicates with the MongoDB database.

The backend follows the **MVC (Model-View-Controller)** architecture.

Models-

Models define the database structure using Mongoose schemas.

Examples:

- User Model
- Driver Model
- Car Model
- Booking Model
- Admin Model

Controllers-

Controllers contain the business logic of the application.

Examples:

- User Controller
- Driver Controller
- Booking Controller
- Cab Controller
- Dashboard Controller

Routes-

Routes define the API endpoints that connect frontend requests to the appropriate controllers.

Examples:

- User Routes
- Driver Routes
- Booking Routes
- Cab Routes
- Dashboard Routes

Middleware-

Middleware is used for authentication and request validation.

The project uses JWT authentication middleware to protect secure APIs.

Responsibilities of the Backend

- Process user requests
- Authenticate users
- Manage bookings
- Manage cabs
- Update ride status
- Generate dashboard statistics
- Communicate with MongoDB

Database Architecture (MongoDB)-

The UCAB system uses **MongoDB**, a NoSQL database, to store application data. MongoDB stores data in collections using JSON-like documents, making it flexible and easy to scale.

The application uses **Mongoose ODM** to interact with the database.

Database Collections

The database contains the following collections:

- Users
- Drivers
- Cars
- Bookings
- Admins

Each collection stores related information and is connected through ObjectId references wherever required.

Responsibilities of the Database

- Store user information
- Store driver details
- Store car information
- Store booking records
- Store administrator data
- Maintain ride history

Working of the Architecture

The working process of the UCAB Cab Booking System is as follows:

1. The user accesses the React frontend through a web browser.
2. The user registers or logs into the application.
3. The frontend sends API requests to the Express.js backend.
4. The backend validates the request and performs the required business logic.

5. The backend communicates with MongoDB using Mongoose.
6. The requested data is retrieved or updated in the database.
7. The backend sends a JSON response to the frontend.
8. The frontend displays the updated information to the user.

For example, when a user books a cab, the booking details are sent from the React frontend to the Express.js backend. The backend validates the request, stores the booking information in MongoDB, and returns a success response. The booking then becomes available for drivers, who can accept or reject the ride. Once accepted, the booking status is updated and displayed to both the user and the administrator.

This architecture provides a clear separation between the user interface, application logic, and database operations, making the UCAB Cab Booking System efficient, maintainable, and scalable for future enhancements.

Setup Instructions:

Prerequisites

Before running the UCAB project, make sure the following software is installed on your computer:

- Node.js
- MongoDB
- Visual Studio Code
- Git
- Postman
- Web Browser (Google Chrome or Microsoft Edge)

These tools are required to run both the frontend and backend of the application.

Installation Steps-

1. Download or clone the project from GitHub.
2. Open the project in Visual Studio Code.
3. Open the terminal and navigate to the **client** folder.
4. Run the following command to install the required packages: **npm install**
5. Open another terminal and navigate to the **server** folder.
6. Install the backend dependencies using: **npm install**

Environment Variables-

Create a **.env** file inside the **server** folder and add the following details:

PORT=8000

MONGO_URI=your_mongodb_connection_string

JWT_SECRET=your_secret_key

These environment variables are used to connect the application to the MongoDB database and provide secure authentication.

Running the Project-

To start the backend server:

cd server

npm start

To start the frontend:

cd client

npm run dev

After both servers are running, open your browser and visit:

<http://localhost:5173>

The UCAB Cab Booking System will now be ready to use.

Folder Structure:

The UCAB project is divided into two main folders: **Client** and **Server**. This separation helps in organizing the frontend and backend code efficiently.

Client Folder-

The **client** folder contains the React.js application. It is responsible for displaying the user interface and interacting with the backend APIs.

Main folders:

- **components/** – Contains reusable React components like Navbar and other UI elements.
- **pages/** – Contains all application pages such as Home, Login, Register, Admin Dashboard, Driver Dashboard, My Bookings, and Book Cab.
- **App.jsx** – Defines application routing.
- **main.jsx** – Entry point of the React application.

Server Folder-

The **server** folder contains the backend developed using Node.js and Express.js.

Main folders:

- **controllers/** – Contains the business logic for users, drivers, bookings, cabs, and dashboard.
- **models/** – Contains MongoDB schemas using Mongoose.
- **routes/** – Defines all API endpoints.
- **middleware/** – Contains authentication middleware using JWT.
- **config/** – Contains MongoDB database connection.
- **server.js** – Main entry point of the backend server.

This folder structure makes the project organized, easy to understand, and easy to maintain. It also follows the MVC (Model-View-Controller) architecture, where each folder has a specific responsibility.

Running the Application:

After installing all the required software and project dependencies, the UCAB application can be started by running both the backend and frontend servers.

Running the Backend-

1. Open the terminal.
2. Navigate to the **server** folder - `cd server`
3. Start the backend server - `npm start`

The backend server will start successfully and connect to the MongoDB database.

Running the Frontend-

1. Open another terminal.
2. Navigate to the **client** folder - `cd client`
3. Start the React application - `npm run dev`

After the frontend starts, open the following URL in your browser:

`http://localhost:5173`

The UCAB Cab Booking System will load successfully, and users can access the application through the web browser.

Application Workflow-

1. Start the MongoDB database.
2. Start the backend server.
3. Start the frontend server.
4. Open the application in the browser.
5. Login as User, Driver, or Admin to access the respective dashboard.

The application is now ready for booking cabs, managing rides, and monitoring the system.

API Documentation:

The UCAB Cab Booking System uses RESTful APIs to enable communication between the frontend and backend. These APIs handle user authentication, cab management, booking operations, and dashboard functionalities.

Method	API Endpoint	Description
POST	/api/users/register	Register a new user
POST	/api/users/login	User login
GET	/api/users/profile	View user profile
POST	/api/drivers/register	Register a new driver
POST	/api/drivers/login	Driver login
GET	/api/cabs/all	View all available cabs
POST	/api/cabs/add	Add a new cab
PUT	/api/cabs/update/:id	Update cab details
DELETE	/api/cabs/delete/:id	Delete a cab
POST	/api/bookings/book	Book a cab
GET	/api/bookings/my	View user bookings
GET	/api/bookings/all	View all bookings (Admin)
PUT	/api/bookings/status/:id	Update ride status
GET	/api/dashboard	View dashboard statistics

API Response-

All APIs return data in **JSON format**. Each response contains a status code and the requested information or an error message if the request fails.

Example:

```
{
  "message": "Booking Successful",
  "status": "Success"
}
```

These APIs provide communication between the React frontend and the Express.js backend, ensuring smooth data transfer throughout the application.

Authentication:

Authentication is used to ensure that only authorized users can access the UCAB Cab Booking System. The project uses **JSON Web Token (JWT)** for secure user authentication and authorization.

When a user, driver, or admin logs in with valid credentials, the server verifies the details and generates a JWT token. This token is stored on the client side and is sent with every protected API request.

The backend uses authentication middleware to verify the token before allowing access to secured routes. If the token is valid, the user can access the requested resources. If the token is invalid or expired, the user is denied access.

Authentication Process

1. User enters email and password.
2. Server verifies the login credentials.
3. A JWT token is generated.
4. The token is stored in the browser.
5. The token is sent in the Authorization header for protected API requests.
6. The server verifies the token before processing the request.

Advantages of JWT Authentication

- Secure user authentication
- Protects private routes
- Prevents unauthorized access
- Fast and lightweight authentication mechanism
- Suitable for MERN Stack applications

The authentication system ensures that users, drivers, and administrators can securely access only the features assigned to their respective roles.

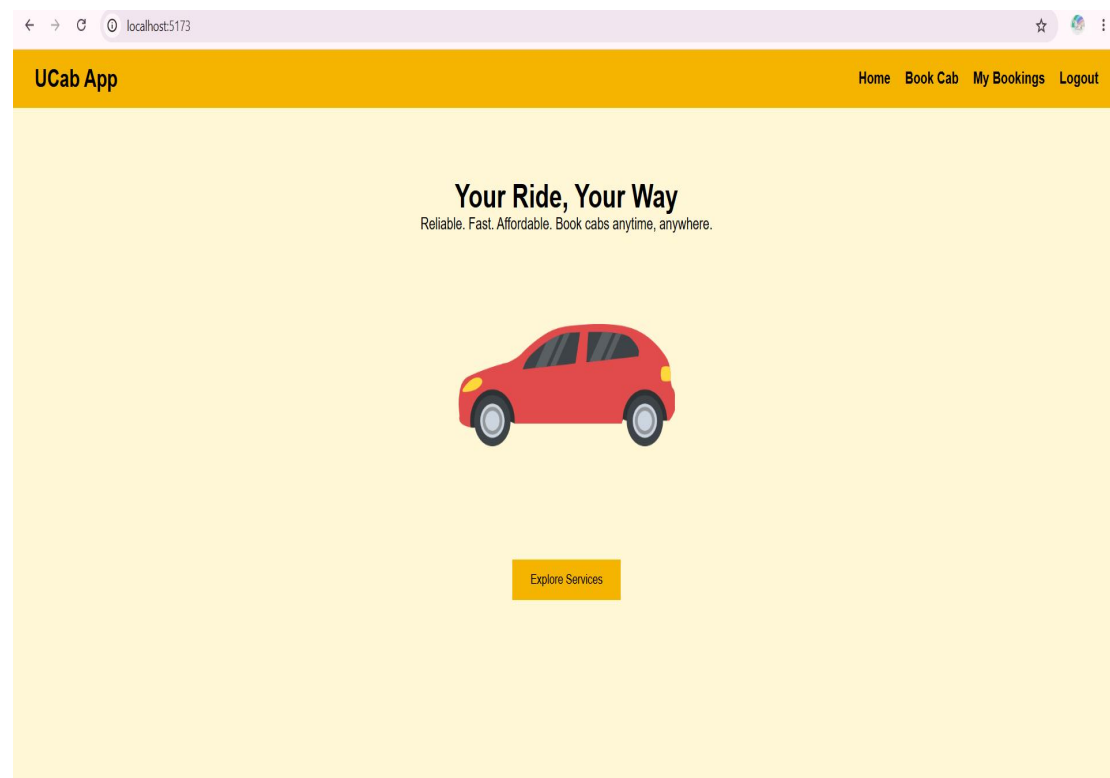
User Interface:

The User Interface (UI) of the UCAB Cab Booking System is developed using **React.js**. It provides a simple, responsive, and user-friendly design that allows users, drivers, and administrators to access the system easily. The interface is designed to ensure smooth navigation and a better user experience.

The application consists of different pages for each type of user.

Home Page-

The Home Page provides an introduction to the UCAB Cab Booking System and allows users to navigate to different sections of the application.



User Registration and Login-

The Registration page allows new users to create an account, while the Login page enables existing users to access their accounts securely using their email and password.

← → ↻ localhost:5173/register

Register

Signup

Already have an account?

Login

← → ↻ localhost:5173/login

Login

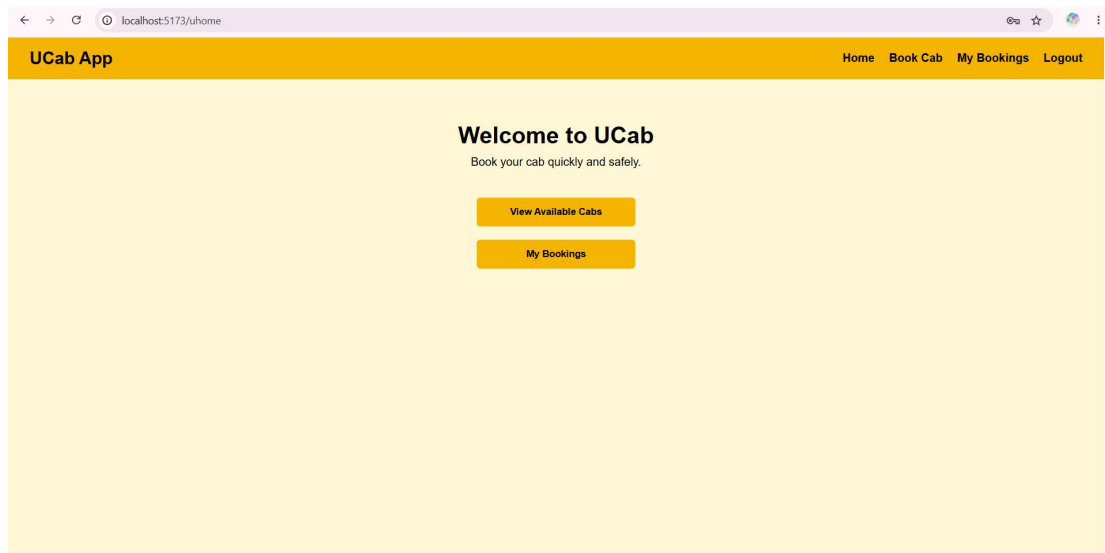
Login

Don't have an account?

Register

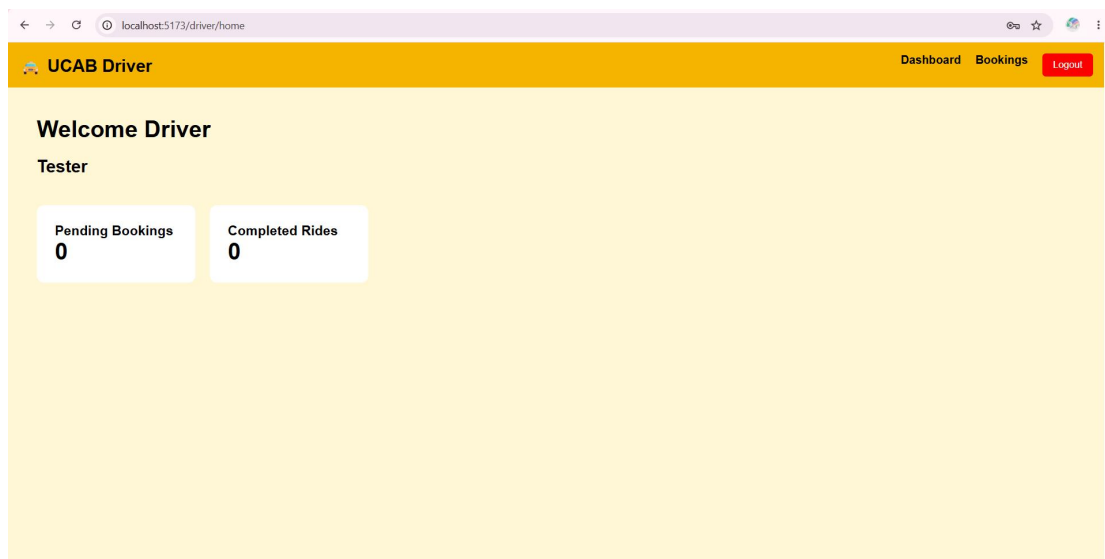
User Dashboard-

The User Dashboard displays the available cabs and allows users to book a ride. Users can also view their booking history and ride status.



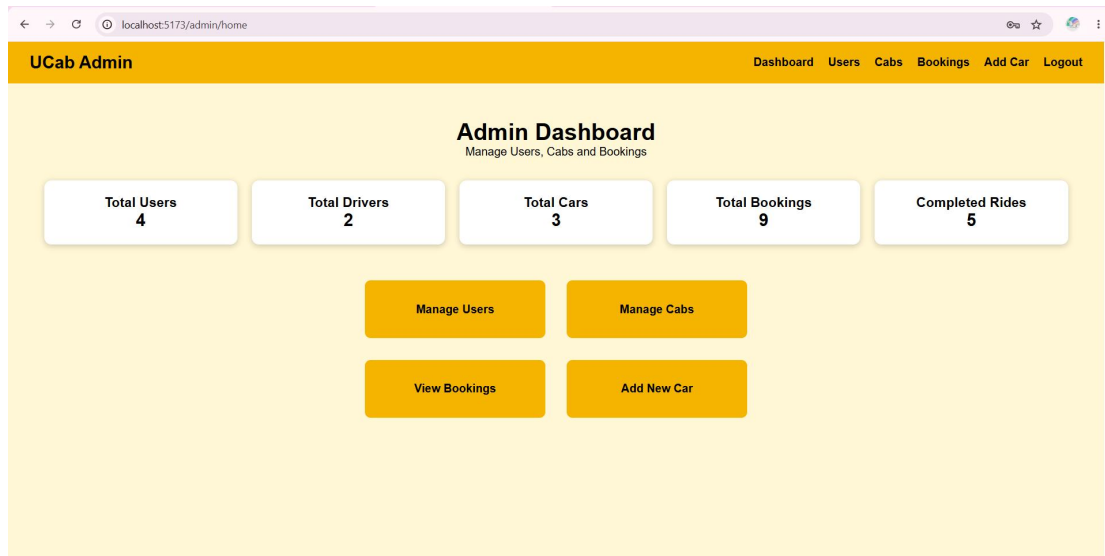
Driver Dashboard-

The Driver Dashboard allows drivers to view assigned bookings, accept or reject ride requests, and update the ride status from Pending to Completed.



Admin Dashboard-

The Admin Dashboard displays important statistics such as Total Users, Total Drivers, Total Cars, Total Bookings, and Completed Rides. It also provides options to manage users, cabs, and bookings.



The user interface is designed to be simple, responsive, and easy to use, ensuring smooth interaction for all users of the system.

Testing:

Testing is an important phase of the UCAB Cab Booking System. It ensures that all modules work correctly and meet the project requirements. The application was tested using **Postman** for backend APIs and **Google Chrome** for frontend functionality.

The following test cases were performed during project testing:

Test Case	Expected Result	Status
User Registration	User account created successfully	Pass
User Login	User logged in successfully	Pass
Driver Login	Driver logged in successfully	Pass
Admin Login	Admin logged in successfully	Pass
View Available Cabs	List of available cabs displayed	Pass
Book a Cab	Booking created successfully	Pass
View My Bookings	User booking history displayed	Pass
Accept/Reject Ride	Driver updates ride request	Pass
Update Ride Status	Ride status updated successfully	Pass
Add New Cab	New cab added successfully	Pass
Dashboard Statistics	Dashboard displays correct data	Pass

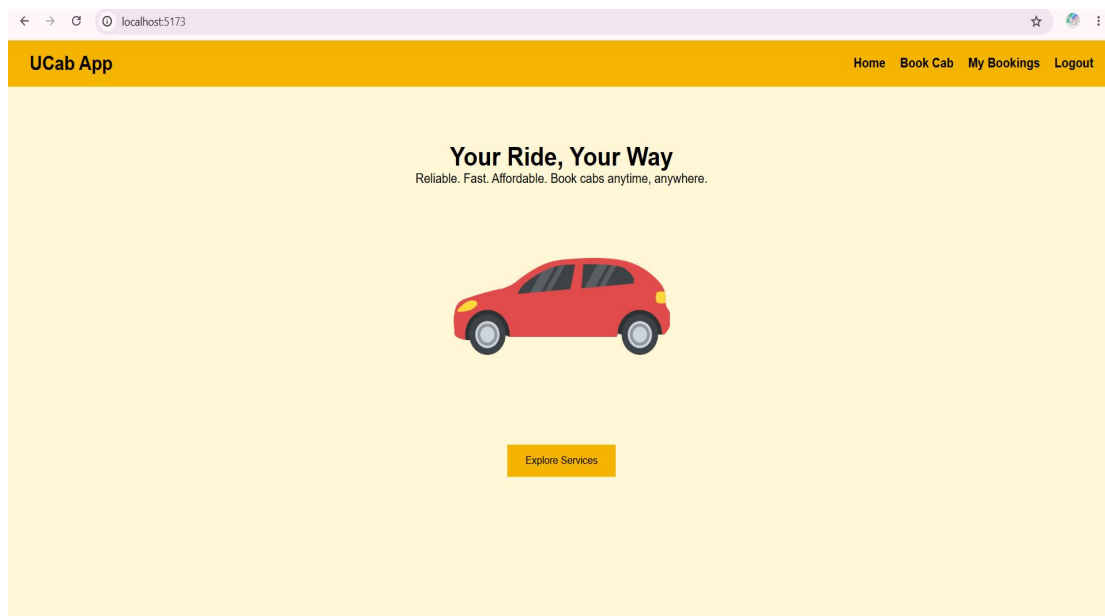
The testing results show that all the major functionalities of the UCAB Cab Booking System are working successfully without any critical errors. The application provides a smooth and user-friendly experience for users, drivers, and administrators.

Screenshots:

The following screenshots demonstrate the major functionalities of the UCAB Cab Booking System. These screenshots show the successful implementation of different modules in the application.

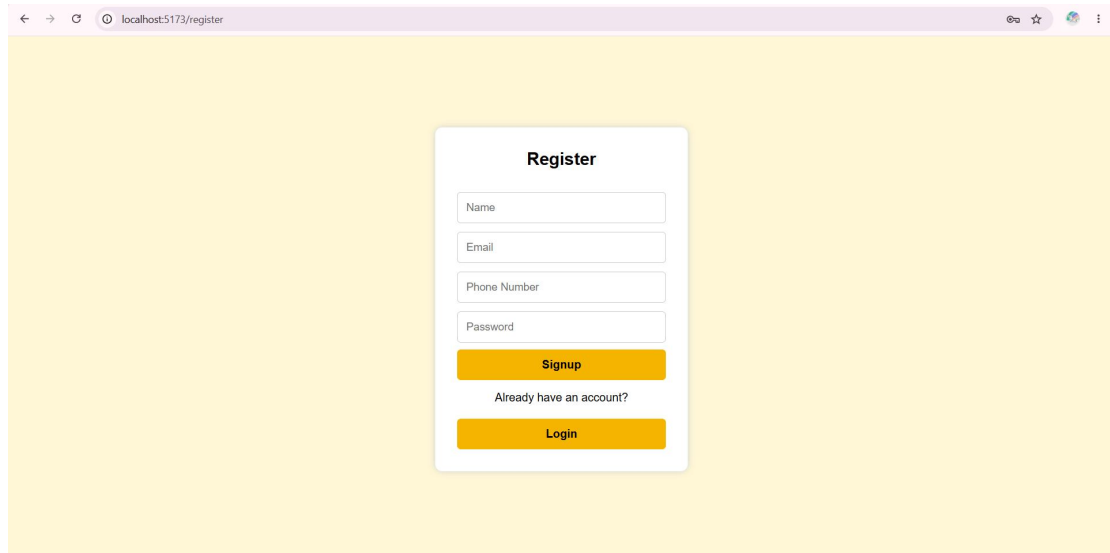
1. Home Page

The Home Page is the first page of the application. It provides navigation options for users, drivers, and administrators.



2. User Registration

This page allows new users to create an account by entering their personal details.

A screenshot of a web browser showing a registration form. The browser's address bar displays 'localhost:5173/register'. The form is centered on a light yellow background and is titled 'Register'. It contains four input fields: 'Name', 'Email', 'Phone Number', and 'Password'. Below these fields is an orange 'Signup' button. Underneath the button is the text 'Already have an account?' followed by an orange 'Login' button.

← → ↻ 📄 localhost:5173/register

Register

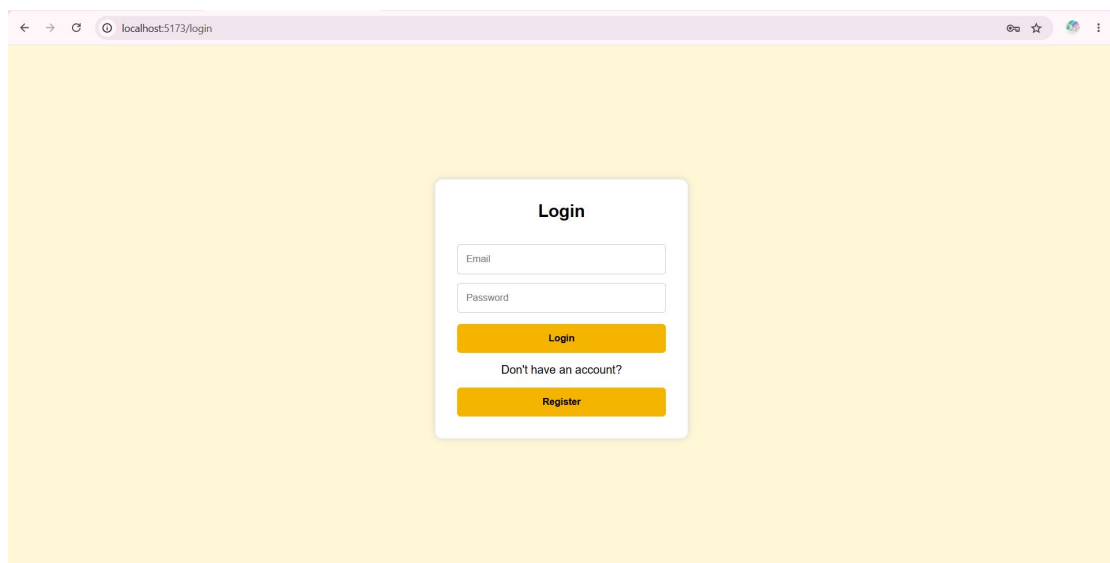
Signup

Already have an account?

Login

3. User Login

This page allows registered users to log in securely using their email and password.

A screenshot of a web browser showing a login form. The browser's address bar displays 'localhost:5173/login'. The form is centered on a light yellow background and is titled 'Login'. It contains two input fields: 'Email' and 'Password'. Below these fields is an orange 'Login' button. Underneath the button is the text 'Don't have an account?' followed by an orange 'Register' button.

← → ↻ 📄 localhost:5173/login

Login

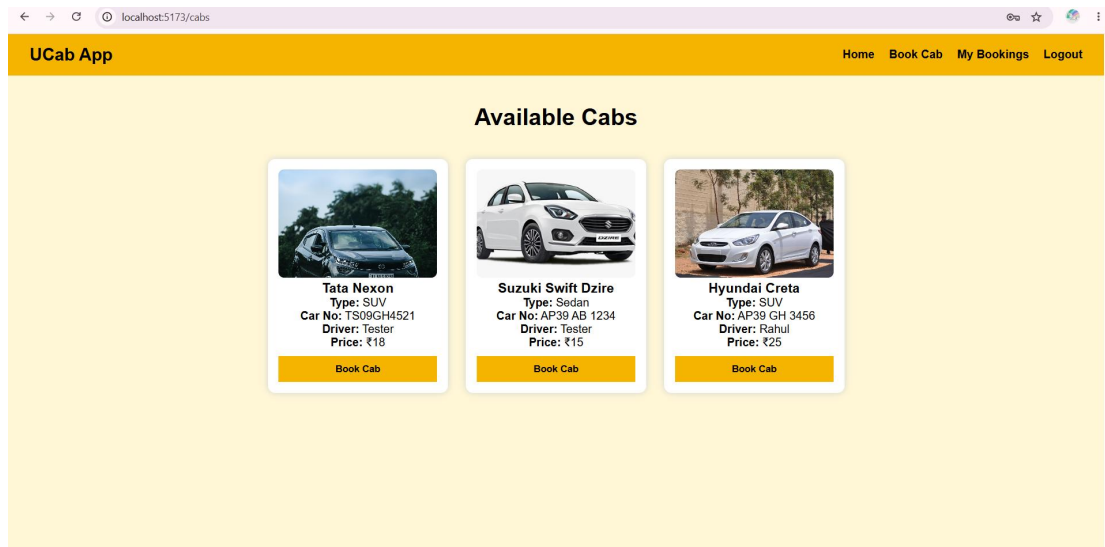
Login

Don't have an account?

Register

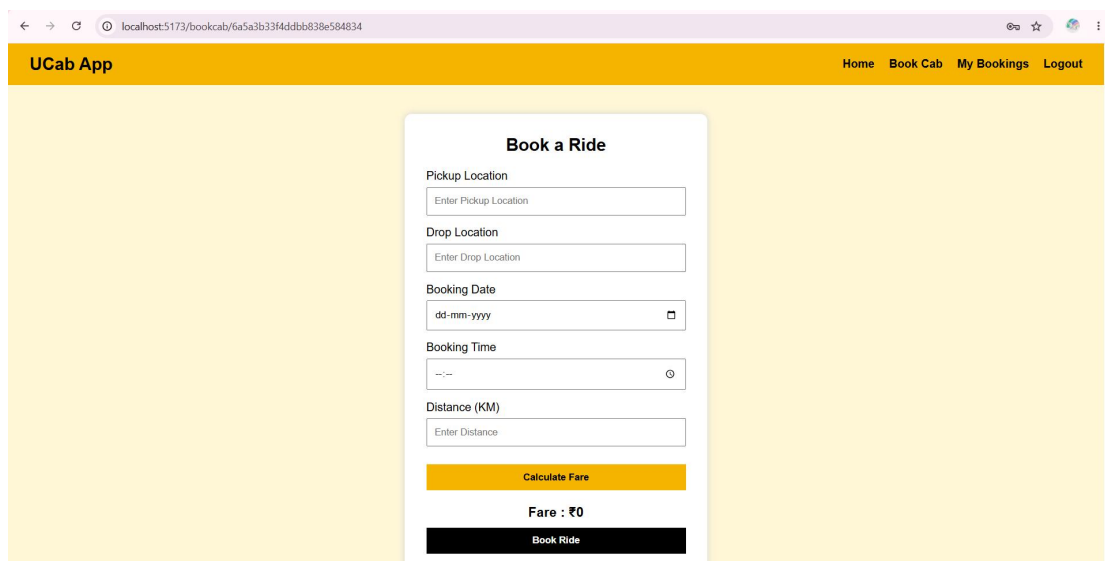
4. Available Cabs

This page displays the list of available cabs along with their details. Users can select a cab and proceed with booking.



5. Book Cab

This page allows users to enter pickup and destination details and confirm the booking.



6. My Bookings

This page displays the booking history and current ride status of the logged-in user.

Cab	Pickup	Drop	Date	Time	Fare	Status	Action	Receipt
Hyundai Creta	hyderabad	vijayawada	-	-	₹500	Completed	✓ Completed	Download
Hyundai Creta	pamar	kanur	-	-	₹150	Cancelled	✗ Cancelled	-
Hyundai Creta	rangoon saheb street	gollapudi	2026-07-18	21:51	₹100	Rejected	✗ Rejected	-
Suzuki Swift Dzire	swathi	gollapudi	2026-07-19	10:00	₹60	Completed	✓ Completed	Download

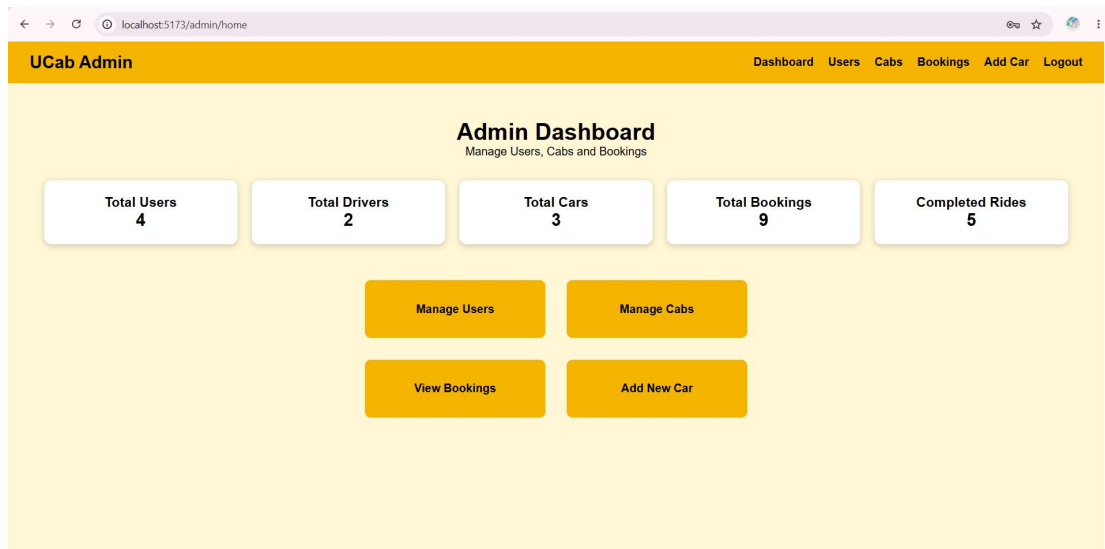
7. Driver Dashboard

The Driver Dashboard allows drivers to view ride requests, accept or reject bookings, and update the ride status.

Welcome Driver	
Tester	
Pending Bookings 0	Completed Rides 0

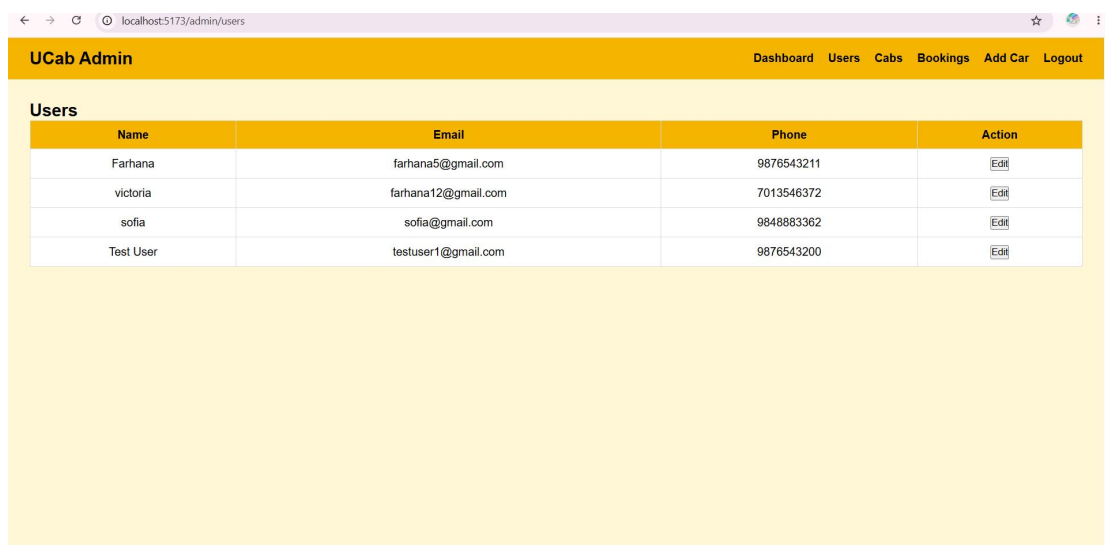
8. Admin Dashboard

The Admin Dashboard displays important statistics such as total users, drivers, cabs, bookings, and completed rides.



9. Manage Users

This page allows the administrator to view and manage all registered users.



10. Manage Cabs

This page allows the administrator to view, add, update, and manage cab details.

Image	Model	Type	Car No	Seats	Price	Action
	Tata Nexon	SUV	TS09GH4521	5	₹18/km	Edit
	Suzuki Swift Dzire	Sedan	AP39 AB 1234	4	₹15/km	Edit
	Hyundai Creta	SUV	AP39 GH 3456	5	₹25/km	Edit

11. View Bookings

This page displays all booking details for administrative monitoring.

User	Cab	Pickup	Drop	Status
Test User	Hyundai Creta	hyderabad	vijayawada	Completed
Test User	Hyundai Creta	pamar	kanur	Cancelled
Test User	Hyundai Creta	rangoon saheb street	gollapudi	Rejected
Test User	Suzuki Swift Dzire	swathi	gollapudi	Completed

The above screenshots demonstrate the successful implementation of the UCAB Cab Booking System and verify that all major modules are functioning correctly.

Known Issues:

Although the UCAB Cab Booking System is fully functional, there are a few limitations in the current version of the project.

- The application does not support live GPS tracking of cabs.
- Online payment functionality is not implemented.
- Email and SMS notifications are not available.
- Driver earnings are not calculated automatically.
- The application currently supports local deployment only and is not deployed on a cloud server.
- Advanced reports and graphical analytics are not included in the current version.

These limitations do not affect the basic functionality of the system. The application successfully performs user registration, cab booking, ride management, and admin operations as intended.

Future Enhancements:

The UCAB Cab Booking System can be enhanced in the future by adding the following features:

- Integration with Google Maps for live cab tracking.
- Online payment gateway using Razorpay or Stripe.
- Email and SMS notifications for booking confirmation and ride updates.
- Driver earnings dashboard with daily and monthly reports.
- Advanced analytics and graphical reports for the admin dashboard.
- OTP-based login and password recovery.
- Push notifications for users and drivers.
- Mobile application development using React Native.
- Cloud deployment for better accessibility and scalability.

These enhancements will improve the overall performance, security, and user experience of the UCAB Cab Booking System.

Conclusion:

The UCAB Cab Booking System is a full-stack web application developed using the MERN Stack (MongoDB, Express.js, React.js, and Node.js). The project provides an efficient platform for users to book cabs, drivers to manage ride requests, and administrators to monitor the overall system.

The application successfully implements user authentication, cab booking, ride management, dashboard analytics, and role-based access control. The project demonstrates the practical implementation of modern web development technologies and follows the MVC architecture for better organization and maintainability.

Overall, the UCAB Cab Booking System meets its objectives by providing a simple, secure, and user-friendly cab booking solution. It can be further enhanced with additional features such as live tracking, online payments, and advanced analytics in future versions.